RepoShield Roadmap

A Simple, Affordable ASPM Platform for Small Engineering Teams

Tejal & Team

September 2026

1. Executive Summary
2. Problem & Market Validation
3. Product Scope
   3.1 In scope (MVP through Year 1)
   3.2 Explicitly out of scope (Year 1)
4. Technical Architecture
   4.1 High-level architecture
   4.2 Frontend
   4.3 Backend
   4.4 Data layer
   4.5 Scanner integration approach
   4.6 LLM integration
   4.7 CI/CD integration
   4.8 Hosting & DevOps
5. One-Year Roadmap
   Phase 1 — Months 1–3: Core MVP
   Phase 2 — Months 3–6: Ownership routing + real users
   Phase 3 — Months 6–9: SLA tracking, CI/CD depth, polish
   Phase 4 — Months 9–12: Launch prep
6. Cost Estimate (Year 1)
7. Differentiation Summary
8. Key Risks

1. Executive Summary

RepoShield is a security posture platform built for small teams and startups — the segment ignored by enterprise ASPM (Application Security Posture Management) vendors like Apiiro, Cycode, and Wiz. Instead of building our own scanning engines or training our own AI model, RepoShield's value comes from three things enterprise tools don't optimize for at small scale:

1. **Fast, cheap aggregation** of existing free/open scanners (Trivy, Semgrep, ZAP, Gitleaks) into one dashboard
2. **Automatic ownership-routing** — findings go straight to the team that owns the code, with no manual handoffs
3. **Setup in under an hour, no sales calls, priced for a 10-person team**

We do not compete with Apiiro/Cycode on enterprise depth (risk graphs, compliance certifications, deep code-to-cloud correlation). We compete on speed, simplicity, and price for teams too small to be worth enterprise vendors' attention.

2. Problem & Market Validation

The problem: small engineering teams run multiple disconnected security tools (dependency scanning, code scanning, live app scanning, secret detection) and manually cross-reference results, then manually message the right developer to fix each issue. This is slow, error-prone, and nobody owns the process.

Why existing tools don't solve this for small teams: - Enterprise ASPM tools (Apiiro, Cycode, Wiz) are priced and built for large organizations — expensive, complex onboarding, sales-call-gated - Individual scanners (Snyk, Semgrep alone) solve one layer, not the cross-tool correlation or team routing problem - Asking a chat LLM (Copilot/ChatGPT) to explain a single finding doesn't solve aggregation, deduplication, or routing at all

Validation step before full build-out: interview 3-5 real small engineering teams or DevOps/security leads. Confirm they (a) use 2+ of these tools already, (b) do NOT already have a lightweight routing/dashboard layer, and (c) would pay for one. If most say they already have this solved informally and it's not painful, revisit scope before continuing.

---

3. Product Scope

3.1 In scope (MVP through Year 1)

- Ingest results from Trivy, Semgrep (free tier), OWASP ZAP, Gitleaks
- Correlate and deduplicate findings across tools
- Risk-rank findings (severity + exploitability heuristics, not a full risk graph)
- Auto-assign findings to owning team via CODEOWNERS file or repo-to-team mapping
- Team-scoped dashboards (each team sees only their own findings)
- LLM-generated plain-language explanations and summary reports (via Gemini/GPT API)
- SLA tracking (time-to-fix per team, overdue alerts)
- CI/CD integration (GitHub Actions first, GitLab CI second)
- Slack/email notifications

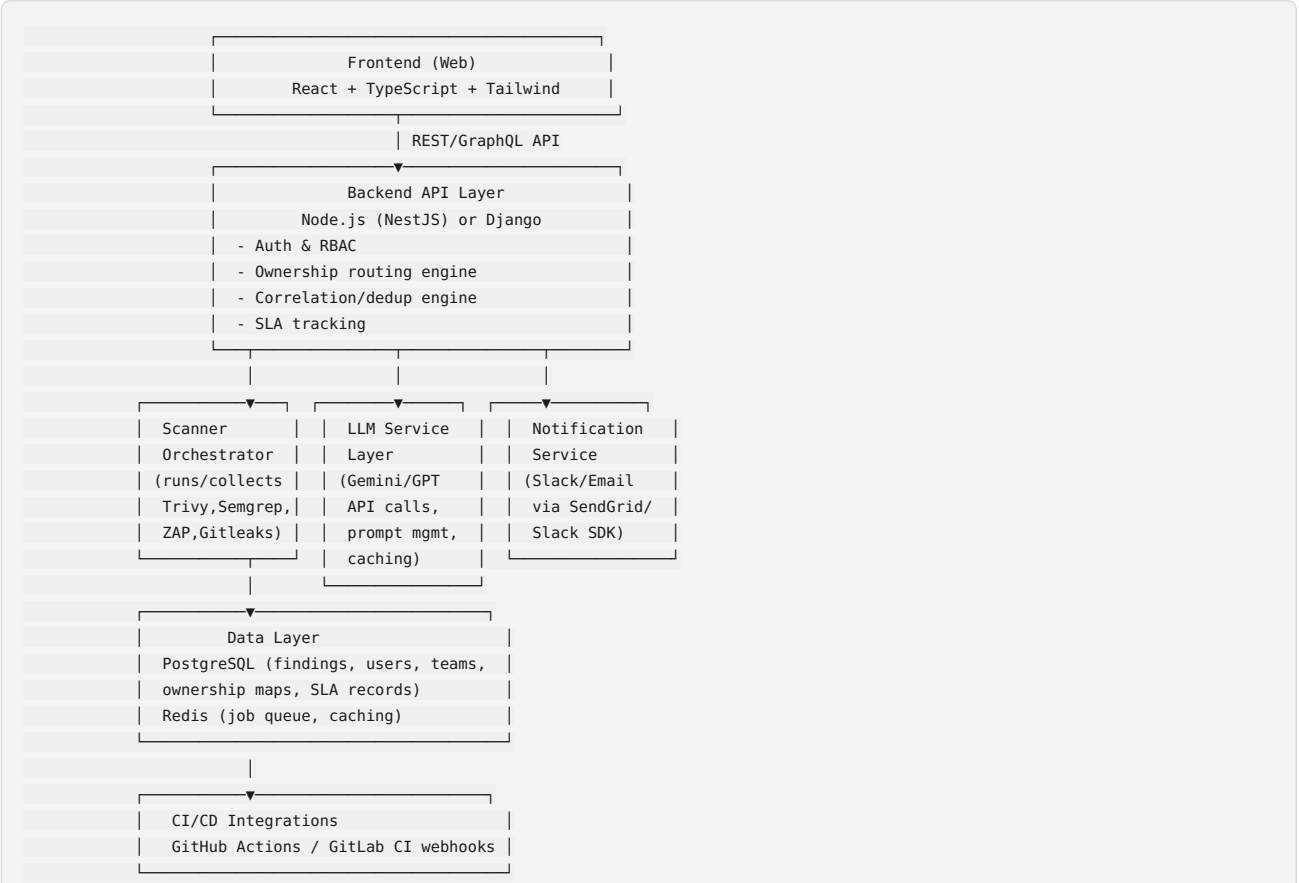
3.2 Explicitly out of scope (Year 1)

- Building our own scanning engines
- Training/fine-tuning our own LLM
- Enterprise compliance certifications (SOC2, FedRAMP) — revisit only once there's paying enterprise demand
- Deep code-to-cloud runtime correlation (Apiiro/Wiz-level graphs)
- On-premise/air-gapped deployment (cloud-only for Year 1)

---

4. Technical Architecture

4.1 High-level architecture



4.2 Frontend

- **Framework:** React + TypeScript, Tailwind CSS for styling
- **Key views:** Org-wide dashboard (security lead view), team-scoped dashboard (developer view), finding detail page, SLA/overdue report, settings (CODEOWNERS mapping, integrations)
- **State/data fetching:** React Query for server state, minimal global state (Zustand) for UI state
- **Auth UI:** handled via Auth0/Clerk SDK (see below)

4.3 Backend

- **Framework:** Node.js with NestJS (or Django REST Framework if the team is stronger in Python) — pick based on team's strongest language
- **API style:** REST for simplicity in Year 1; consider GraphQL later if the frontend needs complex nested queries
- **Core modules:**
  - **Auth & RBAC** — via Auth0 or Clerk (free tier covers early stage), team-based role permissions
  - **Scanner Orchestrator** — triggers Trivy/Semgrep/ZAP/Gitleaks scans (via CLI wrapped in containers) on CI/CD webhook events, or on a schedule
  - **Correlation Engine** — the core IP: normalizes each scanner's output format into one internal schema, matches findings referring to the same underlying issue (e.g. same CVE/library flagged by two tools), and merges them
  - **Ownership Router** — parses CODEOWNERS files or a manual repo-to-team mapping table, assigns each finding to the responsible team automatically
  - **LLM Service Layer** — a thin wrapper around the Gemini/OpenAI API: takes a finding, builds a prompt, calls the API, caches results for identical/similar findings to avoid repeat cost
  - **SLA Tracker** — background job (cron) that checks finding age against team SLA targets, flags overdue items, triggers notifications

4.4 Data layer

- **PostgreSQL** — primary data store: users, teams, repos, findings, ownership mappings, SLA records, audit log
- **Redis** — job queue (via BullMQ if Node.js) for async scan orchestration and LLM calls; also used for caching LLM responses by finding fingerprint

4.5 Scanner integration approach

- Each scanner runs in its own Docker container, triggered by the orchestrator
- Output is parsed from each tool's native format (Trivy: JSON, Semgrep: JSON/SARIF, ZAP: JSON/XML, Gitleaks: JSON) into RepoShield's internal finding schema
- SARIF (Static Analysis Results Interchange Format) is a useful common format — Semgrep and some other tools support it natively, reducing custom parsing work

4.6 LLM integration

- **Model choice:** Gemini 2.5 Flash (or Flash-Lite for simple explanations) for cost efficiency; escalate to a stronger model only for full report synthesis
- **Prompt strategy:** structured prompts with the finding's type, severity, code snippet, and file context; ask for a fixed-format output (explanation + suggested fix) to keep responses consistent and parseable
- **Cost control:** cache responses keyed by a hash of (finding type + code pattern) so near-identical findings across repos don't trigger repeat LLM calls

4.7 CI/CD integration

- **Phase 1:** GitHub Actions — a RepoShield action that triggers scans on PR open/update, posts results as a PR comment, and pushes structured results to the backend
- **Phase 2:** GitLab CI equivalent
- **Webhook-driven:** repo events (PR opened, push to main) trigger the orchestrator via webhook rather than polling

4.8 Hosting & DevOps

- **Hosting:** Start on a single cloud provider (Render, Railway, or a basic AWS/GCP VM) — no need for complex Kubernetes setup at MVP scale
- **CI for RepoShield itself:** GitHub Actions for our own build/test/deploy pipeline
- **Containerization:** Docker for the backend and each scanner tool, docker-compose for local dev
- **Monitoring:** basic uptime + error tracking (Sentry free tier, or simple logging to start)

---

5. One-Year Roadmap

Phase 1 — Months 1–3: Core MVP

- Set up repo, CI/CD for RepoShield itself, basic auth
- Build scanner orchestrator for Trivy + Gitleaks first (simplest, no paid tier needed)
- Build the internal finding schema + basic correlation/dedup logic
- Basic dashboard (list findings, no team routing yet)

- Integrate LLM layer for per-finding explanations
- **Milestone:** a working end-to-end flow — scan a real repo, see deduplicated findings with plain-language explanations in a dashboard

Phase 2 — Months 3–6: Ownership routing + real users

- Add Semgrep and ZAP integration
- Build CODEOWNERS parsing and ownership-routing engine
- Team-scoped dashboard views
- Instrument feedback loop (thumbs up/down on findings, true/false positive marking) — this data becomes valuable later
- Onboard 2-3 pilot teams/friendly early users (student teams, small startups) for real feedback
- **Milestone:** pilot users actively using it on real repos, feedback loop capturing data

Phase 3 — Months 6–9: SLA tracking, CI/CD depth, polish

- SLA tracking + overdue notifications (Slack/email)
- GitHub Actions integration (auto-scan on PR, PR comments)
- Report generation (full LLM-synthesized summary reports, exportable)
- UI/UX polish based on pilot feedback
- Begin exploring GitLab CI integration
- **Milestone:** a genuinely usable product ready to onboard paying small teams

Phase 4 — Months 9–12: Launch prep

- Pricing/billing integration (Stripe)
- Landing page, docs, onboarding flow (self-serve, no sales call required)
- Security review of RepoShield's own infrastructure (practice what we preach)
- Formal launch — outreach to small startup/dev communities, Product Hunt, student/dev networks
- **Milestone:** first paying customers, or confirmed strong pilot conversion if still pre-revenue

---

6. Cost Estimate (Year 1)

Category	Estimated Cost
Trivy, ZAP, Gitleaks	$0 (free, open source)
Semgrep	$0 (free tier for MVP; ~$35–40/developer/month if upgraded later, passed to client pricing)
LLM API (Gemini)	~$1–20/month per active client, scales with usage
Hosting (dev + early production)	~$20–100/month
Auth (Auth0/Clerk free tier)	$0 initially
Domain + misc infra	~$15–30/year
Total estimated spend, Year 1	**Roughly $500–2,000 total**, assuming free tiers cover most of the year

The dominant cost in this project is engineering time, not infrastructure — budget your team's effort accordingly, especially around the correlation engine and ownership router, which are the hardest and most valuable parts to get right.

7. Differentiation Summary

Dimension	Enterprise ASPM (Apiiro/Cycode/Wiz)	RepoShield
Target customer	Large enterprises	Small teams/startups
Setup time	Weeks, sales-call-gated	Under an hour, self-serve
Pricing	Enterprise contracts	Affordable per-team pricing
Ownership routing	Advanced (commit history analysis, active-maintainer detection)	Simple, effective (CODEOWNERS-based)
Compliance depth	Deep (SOC2, FedRAMP, etc.)	Not a focus in Year 1
Core AI use	Proprietary risk graphs	LLM API for explanations/reports only

8. Key Risks

- **Semgrep's free tier detection quality is limited (~45% of issues caught)** — be transparent with early users about this trade-off, or budget for the paid tier as revenue grows
- **Ownership routing via CODEOWNERS is simple but not always accurate** — teams without well-maintained CODEOWNERS files will get weaker routing; plan a fallback (manual mapping UI)
- **Market validation is not yet done** — confirm real demand via user interviews before committing the full year to this scope
- **Competitive response** — this space is active and funded; speed to a usable MVP and real pilot feedback matters more than feature completeness